

아크릴 코드 리뷰 대응 체크리스트

작성일: 2026-05-20

대상: yvvyee/eva-vlm-backend Docker 이미지 및 API 코드

리뷰 출처: 아크릴 1차 회신(ETRI-SDS-VLM-attachments/01-SDS-VLM-detail-for-ETRI.md)

진행 현황 요약

Tier	항목 수	완료	미완료
Tier 0 — 학습 launch 완전 차단	1	✓ 1	0
Tier 1 — 컨테이너 부팅·안정성 필수 수정	7	✓ 7	0
Tier 2 — API 계약·모니터링	5	✓ 5	0
Tier 3 — 인프라·매니페스트 정합	5	⌚ 0	5 (2차 안내 후)
Tier 4 — 결정 사항 (ETRI 선택)	1	✓ 1	0
Tier 5 — ETRI 작업 없음	4	—	—
Tier 6 — 운영 cosmetic (선택)	2	⌚ 1	1 (선택 사항)

Tier 0 — 학습 launch 완전 차단 🚫

A5. PyTorch 보안 취약점(CVE-2025-32434)으로 학습 시작 불가

- 무슨 문제인가?
기존 이미지에 설치된 PyTorch 2.5.1 버전에 보안 취약점이 있습니다. 이 버전에서는 모델 파일을 불러올 때 위험한 코드가 실행될 수 있어, PyTorch 측에서 2.6 버전 미만의 경우 모델 파일 로드 자체를 막아버립니다. 아크릴 클러스터에서 실제로 테스트했을 때, A1~A4 수정을 완료해도 비전 인코더(CLIP 모델) 로드 단계에서 오류가 발생하여 학습이 시작되지 않는 것이 확인됐습니다.
- 어떻게 수정했나?
Docker 베이스 이미지를 CUDA 12.4+Ubuntu 22.04에서 **CUDA 12.8+Ubuntu 24.04**로 업그레이드하고, PyTorch를 **2.11.0** 버전으로 업그레이드했습니다. (2.6 이상이면 해결됩니다.)
- 상태: ☒ 완료 — `eva-vlm-backend:1.1.0` 이미지에 반영

Tier 1 — 컨테이너 부팅·안정성 필수 수정

A1. flash-attn 패키지 누락 — 컨테이너 시작 즉시 오류

- 무슨 문제인가?
에바 VLM은 LLM의 어텐션 연산을 빠르게 처리하기 위해 `flash-attn` 이라는 패키지를 필요로 합니다. 그런데 Docker 이미지를 빌드할 때 이 패키지가 포함되지 않아서, 컨테이너를 실행하면 시작 시점에 바로 오류가 발생하고 추론이 불가능했습니다. 아크릴이 이미지를 직접 실행해서 확인했습니다.
- 어떻게 수정했나?
Dockerfile 에 `flash-attn==2.8.3` 설치 단계를 추가했습니다. 이 패키지는 GPU 드라이버와 함께 소스 빌

드가 필요하므로, 메모리 과부하를 막기 위해 `MAX_JOBS=4` 옵션으로 병렬 빌드 수를 제한합니다.

- 상태:  완료 — `eva-vlm-backend:1.1.0` 이미지에 반영

A2. 학습 상태 영구 고착 버그 (deadlock)

- 무슨 문제인가?

학습을 시작하면 내부적으로 두 가지 작업이 동시에 실행됩니다: ① 실제 학습 프로세스, ② 학습 로그를 읽어서 진행률을 업데이트하는 모니터. 기존 코드에서는 이 두 작업을 `asyncio.gather()` 로 묶었는데, 학습 프로세스가 종료된 뒤에도 모니터가 루프를 빠져나오지 못해서 `GET /train/status` 가 영원히 "running" 상태를 반환하고, 다음 학습도 시작할 수 없게 됩니다. 서버를 재시작하기 전까지 복구가 불가능합니다.

- 어떻게 수정했나?

모니터를 별도 백그라운드 태스크로 실행하고, 학습 프로세스가 종료되면 모니터 태스크를 명시적으로 취소 (`cancel()`) 하도록 수정했습니다. 아크릴이 직접 패치 파일을 작성하여 동작 검증까지 완료한 수정 사항입니다.

- 상태:  완료 — `api/train_manager.py` 반영

A3. 학습 중지 시 GPU를 점유한 자식 프로세스가 남는 버그

- 무슨 문제인가?

예바의 학습에는 DeepSpeed라는 분산학습 프레임워크가 사용됩니다. `POST /train/stop` 으로 학습을 중지하면, 기존 코드는 파이썬 프로세스 하나만 종료 신호를 보냅니다. 그런데 DeepSpeed는 내부적으로 여러 자식 프로세스를 생성하는데, 이 자식들에게는 신호가 전달되지 않아서 GPU 메모리를 점유한 채 계속 살아 있게 됩니다. 이후 새 학습을 시작하면 GPU 메모리가 부족해서 즉시 실패합니다.

- 어떻게 수정했나?

두 가지를 수정했습니다:

1. 학습 시작 시 `start_new_session=True` 옵션으로 프로세스 그룹을 분리합니다.
2. 학습 중지 시 `os.killpg()` 함수로 프로세스 그룹 전체에 종료 신호를 보냅니다.
이렇게 하면 DeepSpeed 자식 프로세스까지 모두 종료되어 GPU가 즉시 해제됩니다.

- 상태:  완료 — `api/train_manager.py` 반영

A4. 동시에 학습 요청이 들어올 때 두 학습이 동시에 시작되는 버그

- 무슨 문제인가?

GPU가 1장인 환경에서는 학습이 1개만 실행되어야 합니다. 기존 코드는 이미 학습 중인지 확인할 때 `job.status == "running"` 조건을 봤는데, 학습 요청이 들어온 직후 수 밀리초 동안은 아직 상태가 "pending" 이라 두 번째 요청이 통과됩니다. 결과적으로 두 학습이 동시에 GPU에서 실행되어 즉시 메모리 부족(OOM) 오류가 발생합니다.

- 어떻게 수정했나?

실행 중인 job이 있는지 확인하는 방식을 `_running_job_id is not None` 으로 변경했습니다. 학습 요청이 접수되는 순간 즉시 `_running_job_id` 가 설정되므로, 이후 요청은 상태와 무관하게 차단됩니다.

- 상태:  완료 — `api/train_manager.py` 반영

D1. 체크포인트 목록 조회가 항상 빈 리스트를 반환하는 버그

- 무슨 문제인가?

GET /models 엔드포인트는 NFS에 저장된 체크포인트(학습 결과물) 목록을 반환해야 합니다. 그런데 코드 내부에서 Path 라는 파이썬 표준 라이브러리를 사용하면서 from pathlib import Path 임포트 문을 빠뜨렸습니다. 실행 시 NameError 가 발생하지만 try/except 로 조용히 처리되어 빈 리스트를 반환하고, 오류가 났다는 사실조차 알기 어렵습니다.

- 어떻게 수정했나?

model_manager.py 파일 상단에 from pathlib import Path 한 줄을 추가했습니다.

- 상태: 완료 — api/model_manager.py 반영

D2. 학습 완료 시 진행률이 100%로 표시되지 않는 버그

- 무슨 문제인가?

학습이 정상 완료되면 진행률이 100%가 되어야 합니다. 기존 코드에서는 job.progress_pct_value = 100.0 으로 처리했는데, progress_pct_value 라는 필드 자체가 TrainJob 클래스에 존재하지 않습니다. 완료 후에도 진행률이 마지막으로 파싱된 값에서 멈춰 있게 됩니다.

- 어떻게 수정했나?

job.current_step = job.total_steps 로 변경했습니다. 진행률은 current_step / total_steps × 100 으로 계산되는 computed property이므로, 이렇게 하면 자동으로 100%가 반환됩니다.

- 상태: 완료 — api/train_manager.py 반영

D3. 학습 파라미터 일부가 실제 학습 스크립트에 전달되지 않는 버그

- 무슨 문제인가?

API로 학습 요청 시 sds_scenario (SDS 시나리오 종류), zero_stage (DeepSpeed ZeRO 최적화 단계) 파라미터를 보낼 수 있습니다. 그런데 기존 코드에서 이 값들이 실제 학습 스크립트 명령어에 포함되지 않아서 사용자가 설정한 값이 조용히 무시됩니다.

- 어떻게 수정했나?

_build_command() 함수에서 zero_stage 값으로 zero2.json 또는 zero3.json 을 선택하도록 수정하고, lora_sds 단계에서 sds_scenario 를 --sds_scenario 인자로 전달하도록 추가했습니다.

- 상태: 완료 — api/train_manager.py 반영

Tier 2 — API 계약·모니터링

B1. 오류 응답의 HTTP 상태 코드가 잘못됨 (항상 200 반환)

- 무슨 문제인가?

웹 서비스의 표준에서는 요청이 실패하면 HTTP 상태 코드로 이를 명확히 알려야 합니다. 예를 들어 "이미 학습 중"이면 429, "해당 job이 없음"이면 404를 반환해야 합니다. 그런데 기존 코드는 모든 상황에서 200(성공)을 반환하고 응답 내용에만 오류를 표시했습니다. 이 방식은 플랫폼의 Kong 게이트웨이가 오류를 감지하지 못해 재시도·모니터링·알림이 정상적으로 동작하지 않습니다.

- 어떻게 수정했나?

각 상황에 맞는 HTTP 상태 코드를 반환하도록 수정했습니다:

- POST /train (이미 실행 중) → 429 Too Many Requests

- GET /train/status (없는 job_id) → **404** Not Found
 - POST /train/stop (없는 job_id) → **404** Not Found
 - POST /train/stop (실행 중 아닌 job) → **409** Conflict
- 상태: ☒ 완료 — api/app.py 반영

B2. /health 엔드포인트가 모델 미로드 상태에서도 200을 반환

- 무슨 문제인가?
쿠버네티스는 /health 엔드포인트를 통해 컨테이너가 요청을 받을 준비가 되었는지 확인합니다(readiness probe). 기존 코드는 모델이 로드되지 않은 상태에서도 항상 200(정상)을 반환해서, 쿠버네티스가 아직 준비되지 않은 Pod에 트래픽을 보내거나, 반대로 liveness probe가 Pod를 불필요하게 재시작시킬 수 있습니다.
- 어떻게 수정했나?
모델이 로드 중이거나 로드되지 않은 경우 **503** Service Unavailable을 반환하도록 수정했습니다. 정상 상태일 때만 200을 반환합니다.
- 상태: ☒ 완료 — api/app.py 반영

B3. 추론과 배포가 동시에 실행될 때 서버가 죽는 버그

- 무슨 문제인가?
POST /run (추론)과 POST /deploy (체크포인트 교체)가 거의 동시에 요청되면 충돌이 발생할 수 있습니다. /deploy 가 기존 모델을 GPU에서 해제하는 순간(self._model = None)에 /run 이 해제된 모델로 추론을 시도하면, 메모리 접근 오류로 서버 프로세스 전체가 죽습니다.
- 어떻게 수정했나?
ModelManager 에 threading.RLock() 을 추가하고, 모델을 교체하는 switch_checkpoint() 와 추론을 실행하는 infer() 가 동시에 실행되지 않도록 직렬화했습니다.
- 상태: ☒ 완료 — api/model_manager.py 반영

B4. 악의적·실수로 인한 대용량 요청에 대한 제한 없음

- 무슨 문제인가?
/run 엔드포인트에 이미지 크기, AIS 데이터 개수, 생성 토큰 수에 대한 상한이 없습니다. 1GB짜리 base64 이미지, 작지만 압축 해제 시 수십 GB가 되는 PNG, max_new_tokens=1000000 같은 입력이 들어오면 서버가 메모리 부족(OOM)으로 죽거나 GPU가 영구 점유됩니다.
- 어떻게 수정했나?
두 가지를 수정했습니다:
 - schemas.py — Pydantic 유효성 검사 제약 추가: image_base64 최대 7MB, ais_rows 최대 64개, max_new_tokens 1~4096 범위
 - model_manager.py — PIL 라이브러리의 decompression bomb 방지(MAX_IMAGE_PIXELS = 32,000,000) 및 EXIF 방향 보정(ImageOps.exif_transpose()) 추가
- 상태: ☒ 완료 — api/schemas.py , api/model_manager.py 반영

B5. TensorBoard 미출력 — 플랫폼 학습 그래프 연동 불가

- 무슨 문제인가?

조나단 플랫폼의 프론트엔드는 학습 중 loss, learning rate 등의 지표를 실시간 그래프로 보여줍니다. 이를 위해 TensorBoard 형식의 로그 파일이 필요한데, 에바의 학습 스크립트는 W&B(Weights & Biases) 또는 "none" 만 사용하고 TensorBoard 출력을 하지 않습니다. 따라서 플랫폼 대시보드에 학습 그래프가 전혀 표시되지 않습니다.

- 어떻게 수정했나?

requirements.txt 에 tensorboard>=2.14 를 추가했습니다. 학습 스크립트(train.py)의 report_to 설정에 "tensorboard" 를 추가하는 작업은 별도로 진행이 필요합니다 (아크릴 제공 패치: train.py.tensorboard.patch 참고).

- 상태: ☒ 완료 — api/requirements.txt 에 tensorboard>=2.14 추가, train.py report_to 에 "tensorboard" 추가 반영

Tier 3 — 인프라·매니페스트 정합 (2차 안내 대기)

아크릴이 플랫폼 탑재 검증 완료 후 2차 안내(5월 말~6월 초)에 확정된 양식을 송부 예정입니다.
현 시점에서는 참고만 하고 구현하지 않습니다.

C1. 멀티테넌시 및 인증 방식 확정 사항 적용

- 내용: 각 워크스페이스마다 별도 Pod 인스턴스 운영. Helm chart에서 Service 타입을 NodePort 에서 ClusterIP 로 변경(NodePort 30080 제거), EVA_WORKSPACE_ID 환경변수 주입 및 검증 미들웨어 추가.
- 상태: ⌚ 2차 안내 후 진행

E1. infer용 + train용 manifest 파일 분리 작성

- 내용: 동일한 Docker 이미지를 두 가지 역할로 나눠 사용. eva-vlm-infer.yaml (항시 상주, 추론 담당)과 eva-vlm-train.yaml (학습 시간 생성)로 분리하여 플랫폼에 등록.
- 상태: ⌚ 2차 안내 후 진행

E2. async 엔드포인트 URL 정책 결정

- 내용: 학습 상태 조회와 중지 URL을 플랫폼 표준(GET /jobs/{id} , POST /jobs/{id}/cancel)으로 변경 할지, 현재 URL(GET /train/status , POST /train/stop)을 유지할지 선택이 필요합니다. 또한 POST /train 응답 코드를 200에서 202 Accepted로 변경 요청.
- 상태: ☒ 완료 — 옵션 A 선택 (플랫폼 표준 URL 채택). api/app.py 에 GET /jobs/{job_id} , POST /jobs/{job_id}/cancel 추가. 기존 URL은 하위 호환 유지. POST /train HTTP 202 변경 완료. 아크릴에 결과 회신 필요.

F1. PVC 마운트 및 환경변수 표준 적용

- 내용: 현재 Helm chart에서 NFS를 직접 마운트하는 방식을 조나단 플랫폼 표준 PVC 마운트로 교체. 환경변수도 EVA_* 에서 JF_* 표준으로 매핑 (JF_MODEL_PATH , JF_DATASET_PATH 등).
- 상태: ⌚ 2차 안내 후 진행

F2. 추론 이미지 EXIF 처리 및 크기 제한

- 내용: B4에서 이미 완료.
- 상태: ☒ 완료

Tier 4 — ETRI 결정 사항 ☒

E2. async 엔드포인트 URL 정책 선택

옵션	URL	ETRI 작업량	비고
✅ (A) 플랫폼 표준 — 선택	GET /jobs/{id} / POST /jobs/{id}/cancel	신규 엔드포인트 추가	기존 URL 하위 호환 유지
(B) 현재 URL 유지	GET /train/status?job_id= POST /train/stop?job_id=	—	—

구현 완료 내용:

- GET /jobs/{job_id} — 학습 진행 조회 (표준 URL) 추가
- POST /jobs/{job_id}/cancel — 학습 중지 (표준 URL) 추가
- POST /train 응답 HTTP 202 Accepted 변경 완료
- 기존 /train/status , /train/stop URL은 하위 호환 유지 (삭제 안 함)

→ 아크릴에 옵션 A 선택 결과 회신 필요

Tier 5 — ETRI 작업 없음 (참고)

항목	내용	처리 주체
C2 Path traversal	checkpoint_name 등 입력 값 경로 검증	F1 완료 시 자동 해결 (아크릴)
C3 DeepSpeed JIT	/root/.cache 쓰기 불가 환경 대비	TORCH_EXTENSIONS_DIR=/tmp/torch_extensions 이미지에 반영 완료 ✅
F3 fine_tuning_job DB	학습 상태 DB 저장	아크릴 측 담당
F4 레지스트리 이전	개인 계정 → 조직 계정 이전	Phase 2 트리거 시 검토

Tier 6 — 운영 cosmetic (선택 사항)

G1. 체크포인트 없을 때 시작 오류 로그 출력 (서비스 영향 없음)

- 내용: 학습 전용 컨테이너처럼 projector.bin 이 없는 환경에서 시작하면 traceback이 출력됩니다. 서비스 동작에는 문제가 없지만 운영자 로그가 지저분해집니다. model_manager.py 에 projector.bin 존재 여부를 사전 확인하고 graceful skip 처리를 추가하면 됩니다.
- 상태: ✅ 완료 — model_manager.py load() 에 사전 존재 확인 후 graceful skip 처리. app.py lifespan도 로드 건너뛰기/실패를 구분하여 출력하도록 수정.

G2. Gradio 데모 앱 hard-coded 경로 수정

- 내용: demo/run.sh 에 개발자 로컬 경로 (/home/ywlee/miniconda3/...)가 하드코딩되어 있고, demo/app.py 에도 로컬 데이터셋·모델 경로가 하드코딩되어 있습니다. 플랫폼 서비스에는 영향이 없지만, ETRI 자체 시연 목적으로 컨테이너 내에서 데모를 실행하려면 수정이 필요합니다.
- 상태: ⌚ 선택 사항 — ETRI 자체 시연 필요 시 진행

Docker 이미지 버전 이력

버전	주요 변경 내용
----	----------

1.0.0	최초 릴리스 — 기본 추론/학습/배포 API
-	CUDA 12.8 + Ubuntu 24.04 + Python 3.12 + PyTorch 2.11.0 + flash-attn 2.8.3 + tensorboard (A1, A5, B5, C3 반영) + A2-A4, B1-B4, D1~D3 코드 수정 반영
1.1.0	E2-A: GET /jobs/{job_id} · POST /jobs/{job_id}/cancel 추가, POST /train HTTP 202 변경, G1 graceful skip 반영 —  푸시 완료 (sha256:d814b1a7)

--	--